— Solution notes —

COMPUTER SCIENCE TRIPOS Part IB – 2022 – Paper 5

7 Introduction to Computer Architecture (swm11)

Memory hierarchy and caching

(*a*) The following code is written in C, where elements within the same row are stored contiguously. Assume each element of the two dimensional arrays A and B is a 64-bit integer. Assume that each cache line is 16-bytes.

```
for(i=0; i<1024; i++)
  for(j=0; j<1024; j++)
    A[i][j] = B[i][0] + A[j][i];
```

(*i*) Which variable references exhibit temporal locality? Explain your answer including a definition of temporal locality. [4 marks]

Answer: Temporal locality means that if a memory location is accessed then it is likely to be used again in the near future. For all iterations of the inner for loop, B[i][0] remains the same so it exhibits strong temporal locality. Additional information: it should be noted that the compiler is likely to optimise the code so that it reads the value once and stores it in a register rather than repeatedly pulling the value from the cache.

In contrast, the locations A[i][j] and A[j][i] are only ever accessed once, so do not exhibit any temporal locality.

(*ii*) Which variable references exhibit spatial locality? Explain your answer including a definition of spatial locality. [4 marks]

Answer: Spatial locality means that if a memory location is accessed then it is likely that its neighbours are going to be accessed shortly after.

A[i][j] accesses exhibit strong spatial locality: when A[i][j] is accessed then for all but the latest iteration of the loop the next item A[i][j+1] will be accessed next. A[j][i] exhibits very weak spatial locality since it strides through the A array. If the A array fits within the cache then we would see the benefit of earlier accesses pulling the array into the cache, but given that A needs to be at least $1024 \times 1024 \times 8bytes = 8MiB$, so we're only going to see spatial locality in the last level cache at best.

Memory hierarchy and caching

(*b*) Imagine we have a tiny 256-byte direct-mapped data cache with 16-byte cache lines. The cache is initially empty. Below is a sequence of 32-bit memory load accesses. For each load identify the tag, index, offset and hit/miss status.

`0x003, 0x0b4, 0x001, 0x102, 0x001, 0x2c2, 0x004, 0x2c0`

[4 marks]

Answer:

tag	index	offset	hit/miss
0x0	0x0	0x3	miss
0x0	0xb	0x4	miss
0x0	0x0	0x1	hit
0x1	0x0	0x2	miss
0x0	0x0	0x1	miss
0x2	0xc	0x2	miss
0x0	0x0	0x4	hit
0x2	0xc	0x0	hit

Pipelining

(*c*) Data hazards can impact the performance of a pipelined processor. What is a data hazard and how can load and arithmetic instructions be reordered to minimise data hazards arising? [4 marks]

Answer: A data hazard typically arises when there is a register write-to-read dependency within the pipeline. For the scalar pipelines we've explored in this course, registers are only written to during writeback at the end of the pipeline. To avoid stale values being read we have to forward recent results within the pipeline or even stall the pipeline until the new values are available. Load-to-use data hazards are handled by forwarding and stalling where necessary. To mitigate stall penalties, the compiler reorders instructions where possible to ensure that load instructions happen as early as possible, a technique called *load hoisting*.

Memory hierarchy and caching

(*d*) For a write-back L1 cache that is full of dirty cache lines, how is a write-miss handled? [4 marks]

Answer: The following steps are required:

- First we determine which cache line we wish to update and then write its "dirty" (i.e. modified) data to the next level in the memory hierarchy.
- For a write-back cache we typically allocate a cache line on a miss, so the vacated cache line is is filled with data from the next level in the memory hierarchy based on the address we wish to write to.
- Then the write updates the cache line and the cache line is marked as dirty.
